

# GravTree: A Local-Compaction Tree for Predictable Write Latency

Subodh Wandile  
Independent Researcher, India  
sawandile\_b17@it.vjti.ac.in

## Abstract

Log-Structured Merge-trees (LSM-trees), the dominant write-optimised storage engine, exhibit tail write latency that is difficult to bound because compaction work is scheduled globally across levels. We present **GravTree**, a tree-structured key-value index in which every node is an immutable Sorted String Table (SSTable) file and compaction is strictly *local*: a single compaction event involves exactly one parent node and its direct children, and nothing else. This invariant bounds the I/O of any single compaction event to  $O(K \cdot B)$  bytes—a constant independent of dataset size—so that the maximum work done between any two writes is fixed by configuration rather than by data volume.

Writes land in an in-memory skip-list at the root; the critical write path performs no disk I/O. A dedicated background thread runs *LocalFlush* to push data from the root buffer down the tree, splitting leaves and internal nodes as they fill. We implement a complete C++17 prototype featuring per-node Bloom filters, a crash-safe atomic manifest with garbage collection of superseded nodes, and a two-level lock design that allows concurrent reads during flushes.

We evaluate the prototype against RocksDB 11.8 at  $N = 10\text{M}$  operations per workload under equal durability settings (no WAL on either engine). GravTree achieves  $1.3\text{--}2.6\times$  lower p99 latency on all four workloads and a predictable tail: p99.99 write latency stays below  $20\ \mu\text{s}$ , versus  $31\text{--}2117\ \mu\text{s}$  for RocksDB. Point reads are  $2.6\times$  faster at p99 and  $2.4\times$  higher in throughput. The price is write throughput: the current single-threaded flush design rewrites whole nodes on every *LocalFlush* and sustains  $21\text{--}80\times$  fewer writes per second than RocksDB. We quantify this trade-off, and discuss how GravTree’s immutable, sequentially-written node files are structurally compatible with Zoned Namespace (ZNS) SSDs.

## 1 Introduction

Write-optimised storage engines power the data layer of cloud databases, time-series stores, and messaging systems. The dominant design, the Log-Structured Merge-tree (LSM-tree) [13], achieves high write throughput by buffering writes in memory and periodically merging sorted runs on disk. Systems including RocksDB [10], LevelDB [9], Cassandra, and TiKV all build on this foundation.

The central cost of the LSM design is *compaction*: a background process that reads files from level  $L_i$ , merges them with overlapping files from level  $L_{i+1}$ , and writes out a new set of  $L_{i+1}$  files. Compaction can cascade—a newly enlarged  $L_{i+1}$  triggers further compaction into  $L_{i+2}$ , and so on—so the I/O cost of a single compaction event is proportional to the sizes of all affected levels. When the write rate outpaces compaction throughput, RocksDB stalls or throttles incoming writes until compaction catches up. These stall events are well-documented in production deployments [4, 8]: p99.9 write latency routinely spikes  $100\times\text{--}1000\times$  above median.

Efforts to tame compaction have focused on reducing *write amplification* [7, 8, 12, 14] or restructuring the level hierarchy [14], but none bound the *duration* of a single compaction event to a constant independent of dataset size.

**GravTree** is built around a single invariant that does exactly this:

*A LocalFlush involves exactly one parent node and its direct children—no other nodes are read or written.*

Every node in GravTree is an immutable SSTable file. Incoming writes accumulate in an in-memory skip list at the root. When the buffer exceeds a configurable soft threshold, a background thread runs *LocalFlush*: it partitions the buffer by child key ranges, merge-sorts each partition with the corresponding child’s

on-disk data, writes new immutable child files, and invalidates the old ones. The total I/O per flush is  $O(K \cdot B)$  bytes where  $K$  is the fanout ( $\leq 8$ ) and  $B$  is the buffer threshold—a bounded constant. Writers block only if the buffer reaches a hard limit of twice the threshold before the flush completes.

GravTree is closest in spirit to the  $B^\epsilon$ -tree [1] and its high-performance descendant SplinterDB [6], which also propagate writes downward through per-node buffers. The key difference is our storage model: every GravTree node is a complete, immutable SSTable (not a mutable fixed-size block). This enables per-node Bloom filters with no staleness concern, deterministic crash recovery via atomic file rename, and a natural mapping onto ZNS SSDs where each node occupies one sequentially-written zone.

### Contributions.

1. The GravTree data model and LocalFlush algorithm, with a formal bound on per-flush I/O cost and a recursive split rule that keeps every node within a configured size as the tree grows (§3).
2. A complete C++17 implementation with skip-list write buffer, per-node Bloom filters, sparse index, CRC-64 integrity, atomic manifest, and manifest-driven garbage collection, validated at 10M keys for completeness and crash-restart consistency (§4).
3. An empirical evaluation at  $N = 10M$  operations against RocksDB 11.8 showing 1.3–2.6 $\times$  lower p99 latency on all workloads and a p99.99 write tail below 20  $\mu$ s, together with an honest measurement of the write-throughput and write-amplification cost of the design (§5).
4. A structural analysis of why GravTree’s immutable node model is a natural fit for ZNS SSDs (§6).

## 2 Background and Motivation

### 2.1 LSM-Trees

An LSM-tree organises data as a sequence of levels  $L_0, L_1, \dots, L_k$ . Writes accumulate in an in-memory MemTable; when full, the MemTable is flushed as an immutable SSTable to  $L_0$ . When  $L_0$  accumulates enough files, a compaction job merges them with overlapping  $L_1$  files to produce new  $L_1$  files. This repeats recursively: a sufficiently large  $L_i$  triggers compaction into  $L_{i+1}$ .

The total I/O cost of a compaction at level  $L_i$  is proportional to the sizes of all files read and written at levels  $L_i$  and  $L_{i+1}$ , which can be several GB. When the write rate exceeds the compaction throughput on the available I/O bandwidth, RocksDB imposes a write delay (soft limit) or halts writes entirely (hard limit) until compaction catches up. The resulting latency spikes are unbounded in duration—they last as long as the I/O subsystem needs to drain the backlog.

### 2.2 $B^\epsilon$ -Trees

The  $B^\epsilon$ -tree [1] routes writes as “messages” through internal node buffers, batching them before pushing to children. A flush operation moves a buffer’s worth of messages one level down, amortising I/O across many logical writes. The flush scope is bounded to a single parent-child edge, which is the same locality property exploited by GravTree.

SplinterDB [6] is the state-of-the-art  $B^\epsilon$ -tree implementation, achieving write throughput competitive with RocksDB while offering better tail latency. GravTree differs from SplinterDB in two ways: (1) nodes are immutable SSTable files rather than mutable B-tree pages, so crash recovery requires only an atomic manifest update and no write-ahead log for tree structure; and (2) per-node Bloom filters are always consistent with the on-disk content, since a node’s entries never change after it is written.

### 2.3 The Stall Characterisation Problem

A key question is: *what bounds the worst-case duration of a single compaction event?* For an LSM-tree at level  $L_i$  with  $S_i$  bytes of data, the answer is  $\Theta(S_i + S_{i+1})$ , which grows with the dataset. For GravTree, the answer is  $O(K \cdot B)$  regardless of dataset size. This is the fundamental architectural difference that GravTree exploits.

### 2.4 Zoned Namespace SSDs

ZNS SSDs [3] replace the conventional block interface with a set of *zones* that must be written sequentially and explicitly reset before reuse. This eliminates the device-level Flash Translation Layer (FTL) and its associated garbage collection overhead. Existing storage engines adapted for ZNS (e.g., ZenFS [2]) still inherit the multi-level compaction behaviour of their host engine. GravTree’s append-only node model aligns structurally with ZNS requirements without requiring an adaptation layer.

---

**Algorithm 1** Insert(*key*, *value*)

---

```
1: if buf.bytes() ≥ hardLimit then
2:   WAITFORFLUSH    ▷ back-pressure; rarely
   taken
3: end if
4:  $e \leftarrow \langle \text{key}, \text{value}, \text{NextSeq}(), \text{false} \rangle$ 
5: buf.Insert( $e$ )      ▷  $O(\log n)$ , no disk I/O
6: if buf.bytes() ≥ softThreshold then
7:   SIGNALFLUSHTHREAD
8: end if
```

---

## 3 GravTree Design

### 3.1 Data Model

GravTree is a  $K$ -ary tree of *nodes*. Each node  $v$  stores:

- A non-negative *level*  $\ell(v)$ ; level 0 denotes a leaf.
- An immutable *SSTable file* containing a sorted sequence of entries  $\langle \text{key}, \text{value}, \text{seq}, \text{deleted} \rangle$ .
- References to at most  $K$  child nodes, each covering a disjoint key sub-range.
- A Bloom filter and sparse index embedded in the SSTable file.
- A *key range*  $[\text{min\_key}, \text{max\_key}]$  covering all keys in the subtree rooted at  $v$ .

The *root node* additionally maintains an in-memory skip-list write buffer that absorbs all incoming writes without disk I/O. A global *manifest* file records the set of valid nodes and the current root ID.

### 3.2 Write Path

Algorithm 1 shows the write path. The only critical-path work is a skip-list insert ( $O(\log n)$ ) into DRAM. A dedicated background thread is signalled when the buffer exceeds a soft threshold (default 4MB); the thread drains the buffer without blocking writers. Writers block only at the hard limit (default  $2 \times$  soft threshold), which is never reached under normal sustained write rates.

The skip list orders entries by (*key* asc, *seq* desc), so the most recent version of a key is always the first encountered during lookup.

### 3.3 LocalFlush

Algorithm 2 is the core operation. Its I/O cost is bounded:

---

**Algorithm 2** LocalFlush(*root*  $r$ )

---

```
1:  $E \leftarrow \text{buf.ToSortedVector}(); \text{buf.Clear}()$ 
2: Partition  $E$  into  $E_1, \dots, E_K$  by child key ranges
3: for  $i \leftarrow 1$  to  $K$  do
4:    $E'_i \leftarrow \text{Merge}(E_i, \text{Load}(c_i))$     ▷ two-way
   merge-sort
5:    $c'_i \leftarrow \text{WriteSSTable}(E'_i)$  ▷ new immutable file
6:   Invalidate( $c_i$ )    ▷ old file now garbage
7: end for
8: Update child pointers in  $r$ 
9: AtomicManifestSave()
```

---

**Proposition 1.** *A single LocalFlush reads and writes at most  $B + K \cdot C_{\max}$  bytes, where  $B$  is the root buffer size and  $C_{\max}$  is the size of the largest child SSTable. Both  $B$  and  $C_{\max}$  are bounded by configuration constants independent of the total dataset size  $N$ .*

**Merge semantics.** Entries from the write buffer and from a child SSTable are merged by (*key* asc, *seq* desc). For any key, only the entry with the highest *seq* is retained in the output. A tombstone entry (*deleted* = true) suppresses all earlier versions of its key.

### 3.4 Leaf Split

When the root is a childless leaf and its buffer reaches the soft threshold, a leaf split creates two new leaf SSTables and a new internal root:

1. Sort the write buffer; choose the median key as the fence key.
2. Write the left and right halves as two new leaf SSTable files.
3. Create a new internal root referencing both leaves.
4. Atomically update the manifest.

The old leaf file is invalidated and can be reclaimed.

### 3.5 Read Path

The read path (Algorithm 3) first checks the in-memory skip list in  $O(\log n)$ . If not found, it traverses the root-to-leaf path, using each node's Bloom filter to skip nodes unlikely to contain the key, then performs a binary search within the node's sparse-indexed SSTable.

---

**Algorithm 3**  $\text{Get}(key)$ 

---

```

1: if  $e \leftarrow \text{buf.FindKey}(key)$  then  $\triangleright O(\log n)$ ,  
   DRAM only  
2:   return  $e.deleted ? \perp : e.value$   
3: end if  
4:  $path \leftarrow \text{FindPath}(key)$   $\triangleright$  root-to-leaf traversal  
5: for each node  $v$  in  $path$  do  
6:   if  $v.MightContain(key)$  then  $\triangleright$  Bloom filter  
7:     if  $e \leftarrow v.BinarySearch(key)$  then  
8:       return  $e.deleted ? \perp : e.value$   
9:     end if  
10:  end if  
11: end for  
12: return  $\perp$ 

```

---

### 3.6 Complexity

Let  $n$  be the number of entries in the write buffer,  $B$  the buffer threshold in bytes,  $K$  the fanout, and  $H = O(\log_K(N/B))$  the tree height for a dataset of  $N$  entries.

- **Write (amortised):**  $O(\log n)$  — skip-list insert, DRAM only.
- **LocalFlush I/O:**  $O(K \cdot B)$  bytes — bounded constant (Proposition 1).
- **Point read:**  $O(H \cdot \log n_v)$  disk reads in the worst case, where  $n_v$  is the number of entries per node; Bloom filters reduce the effective constant dramatically in practice.
- **Write amplification:** Each entry is rewritten once per tree level it descends through, giving write amplification  $\leq H = O(\log_K(N/B))$ —the same asymptotic bound as a  $B^\epsilon$ -tree.

## 4 Implementation

The GravTree prototype is approximately 1,500 lines of C++17, open-sourced under the AGPL license. We describe the key implementation decisions.

### 4.1 Skip-List Write Buffer

The write buffer is a probabilistic skip list with maximum height 12 and branching probability  $p = 1/4$ , matching the parameters used in LevelDB’s MemTable [9]. Compared to a sorted `std::vector`, the skip list eliminates the  $O(n)$  element-shifting cost on insert, which was the dominant bottleneck in our earlier vector-based prototype and caused a  $5\times$  write latency disadvantage at high buffer occupancy.

### 4.2 SSTable File Format

Each node file is written once and never modified. Its layout is:

| Section                                                  | Size     |
|----------------------------------------------------------|----------|
| Magic number + version                                   | 12 B     |
| Header (id, level, num_children, ...)                    | 64 B     |
| Child ID array ( $\leq 16$ entries)                      | 128 B    |
| Bloom filter (serialised bit-array)                      | variable |
| Sorted entries (key, value, seq, deleted)                | variable |
| Sparse index (every $128^{\text{th}}$ key + file offset) | variable |
| CRC-64 checksum (covers all preceding bytes)             | 8 B      |

Files are written atomically via a temporary file renamed into place, ensuring that a partially-written file is never visible as a valid node.

### 4.3 Bloom Filters

Each on-disk node carries a Bloom filter targeting a 1% false-positive rate, built using MurmurHash3-128 with the Kirsch-Mitzenmacher double-hashing technique [11]. Filters are serialised into the SSTable file and loaded lazily on first access.

One subtlety: the root node’s in-memory write buffer grows after the Bloom filter was last serialised, so the filter may not reflect recently buffered keys. We handle this by bypassing the Bloom check for any node that has a non-empty in-memory buffer, falling back to a full binary search. Once a node is flushed and its buffer is empty, the Bloom filter is authoritative.

### 4.4 Manifest and Crash Safety

A text-format manifest records: the set of valid node files with their IDs, levels, and paths; the current root ID; and a monotonically increasing next-node-ID counter. All manifest mutations are performed via write-to-temp + rename, so the manifest is always either the old or new state—never partially updated.

On restart, GravTree reads the manifest, ignores any node file whose CRC-64 check fails, and loads the root node into the in-memory node cache.

**Durability limitation.** The current prototype does not maintain a write-ahead log (WAL) for the in-memory write buffer. A crash before `flush()` is called will lose any writes buffered since the last flush. Adding a WAL or making the skip list itself persistent (e.g., via NVM) is straightforward future work.

## 4.5 Concurrency Model

GravTree uses a two-level lock hierarchy to separate the hot write path from the tree-structure path:

- `write_buf_mu` (`std::mutex`): guards the skip-list write buffer. Held only for the duration of a skip-list insert— $O(\log n)$  DRAM operations, no disk I/O. This is the only lock on the critical write path.
- `cache_mu` (`std::shared_mutex`): guards the node cache and tree structure. The background flush thread acquires an exclusive lock while modifying tree structure; concurrent readers (`get`, `range_scan`) hold a shared lock, permitting parallel reads.

The two locks are never held simultaneously, ruling out deadlock. The flush thread releases `cache_mu` before writing SSTable files to disk, so disk I/O does not block readers.

## 5 Evaluation

### 5.1 Experimental Setup

**Hardware.** All experiments run on a Linux x86-64 virtual machine with 8 vCPUs (Intel Xeon Platinum 8559C), 31 GB RAM, no swap, and a 124 GB NVMe-backed virtual disk (ext4), running Ubuntu 22.04.5 LTS. Both engines write to the same filesystem.

**Software.** GravTree is built with GCC in Release mode (`-O3`). We compare against RocksDB 11.8.1 built from source with the same compiler.

**Configuration.** Both engines use a 4 MB write buffer. RocksDB is configured with `kNoCompression`, `max_write_buffer_number = 3`, one background compaction thread and one background flush thread. GravTree uses fanout  $K = 8$  (`MAX_FANOUT = 16` for internal nodes) and hard-limit factor 2.0. *Durability is equalised*: the GravTree prototype has no write-ahead log, so RocksDB is run with `disableWAL = true`. Both engines therefore offer the same guarantee (buffered writes are lost on crash) and neither pays a per-write log `fsync`.

**Methodology.** Each workload performs  $N = 10,000,000$  operations with 100-byte values and 20-byte zero-padded decimal keys. Latency is measured per-operation with `std::chrono::high_resolution_clock`; results are reported as sorted percentiles with no warm-up exclusion. Throughput is wall-clock operations per

Table 1: Write latency ( $\mu\text{s}$ ) and throughput at  $N = 10\text{M}$ . **Bold** = lower latency. Ratio  $> 1$  favours GravTree.

|                      | p50         | p99         | p99.9             | p99.99            | Mops/s       |
|----------------------|-------------|-------------|-------------------|-------------------|--------------|
| <i>SeqWrite</i>      |             |             |                   |                   |              |
| GravTree             | <b>0.26</b> | <b>0.80</b> | <b>1.72</b>       | <b>9.2</b>        | 0.067        |
| RocksDB              | 0.62        | 1.28        | 5.45              | 31.4              | <b>1.439</b> |
| Ratio                | 2.4×        | 1.6×        | 3.2×              | 3.4×              | 0.047×       |
| <i>RandWrite</i>     |             |             |                   |                   |              |
| GravTree             | <b>0.72</b> | <b>2.43</b> | <b>7.21</b>       | <b>17.3</b>       | 0.003        |
| RocksDB              | 1.08        | 3.20        | 1061.6            | 2116.7            | <b>0.232</b> |
| Ratio                | 1.5×        | 1.3×        | 147× <sup>‡</sup> | 122× <sup>‡</sup> | 0.013×       |
| <i>Mixed (50/50)</i> |             |             |                   |                   |              |
| GravTree             | <b>1.54</b> | <b>4.33</b> | <b>9.91</b>       | <b>19.2</b>       | 0.103        |
| RocksDB              | 2.43        | 9.81        | 17.25             | 83.0              | <b>0.309</b> |
| Ratio                | 1.6×        | 2.3×        | 1.7×              | 4.3×              | 0.33×        |

<sup>‡</sup>RocksDB’s RandWrite p99.9/p99.99 are dominated by a single write stall (1–4 ms per operation) confined to the final  $\approx 40\text{K}$  operations; see §5.5.

second over the timed loop *including* the closing `flush()`, so it reflects the total cost of making all  $N$  writes persistent. Before benchmarking, a separate verification harness inserted 1M keys, read every key back, closed the engine, re-opened it from the manifest, and read every key again; all reads returned the correct value (0 missing, 0 mismatched).

### 5.2 Workloads

- **SeqWrite**: Keys  $0, 1, \dots, N-1$  in order.
- **RandWrite**: Keys drawn uniformly at random from  $[0, 2N)$ .
- **PointRead**: Dataset pre-filled with  $N/10 = 1\text{M}$  sequential keys ( $\approx 120\text{ MB}$ ,  $30\times$  the write buffer, so most reads go to disk);  $N$  random point lookups.
- **Mixed**:  $N/2$  sequential keys pre-inserted;  $N$  operations alternating random reads and appending writes.

### 5.3 Write Latency

Table 1 shows write latency and throughput. At p50 GravTree is  $1.5\text{--}2.4\times$  faster: a write is a skip-list insert into DRAM ( $\approx 0.26\ \mu\text{s}$ ), whereas RocksDB’s memtable insert plus write-group coordination costs  $\approx 0.6\text{--}1.1\ \mu\text{s}$ . With the WAL disabled on both engines this gap is purely in-memory overhead, not durability cost.

At p99, GravTree’s advantage is  $1.3\text{--}2.3\times$ . At

p99.9 and p99.99 the picture is qualitatively different: GravTree’s tail stays below  $20\mu\text{s}$  on every write workload, whereas RocksDB’s p99.99 ranges from  $31\mu\text{s}$  (SeqWrite) to  $2.1\text{ms}$  (RandWrite). The RandWrite outlier is a RocksDB write stall, not steady-state behaviour: with a 4MB memtable and `max_write_buffer_number = 3`, the writer eventually outruns the single flush thread and is throttled. GravTree’s writer is throttled too (the hard limit at twice the buffer threshold blocks `insert()` until LocalFlush completes), but that back-pressure is exercised rarely at this scale and its cost lands on a small number of operations.

**Throughput.** The right-hand column is the cost of the design. RocksDB sustains  $21\times$  (SeqWrite) to  $80\times$  (RandWrite) more writes per second. Two factors dominate. First, every LocalFlush rewrites whole nodes: a 4MB root flush that touches  $c$  children rewrites up to  $c \cdot B$  bytes, and when a child’s own buffer fills it recursively rewrites its children. For uniformly random keys every root flush touches all children, so write amplification grows with the number of tree levels and the fanout. We measured it directly (bytes passed to `write(2)` by the process, from `/proc/self/io`, divided by the  $N \cdot 120\text{B}$  logical payload; background compactions drained before reading the counter). At 1M operations GravTree writes  $6.5\times$  (SeqWrite) and  $8.6\times$  (RandWrite) the logical volume versus  $1.0\times$  and  $3.3\times$  for RocksDB. At 10M operations SeqWrite stays at  $9.4\times$  (RocksDB  $1.0\times$ ), but RandWrite reaches  **$180\times$**  (216GB written for 1.2GB of payload) versus  $10.0\times$  for RocksDB: with uniformly random keys every root flush is spread across all children, so each flush rewrites essentially the whole tree below it, and the rewrite volume grows with the dataset size. This, not per-operation overhead, is why the 10M RandWrite run took 51 minutes. Second, the flush thread is single-threaded and performs merge, Bloom-filter construction, CRC-64 and file write serially while holding the tree lock. The single-threaded flush is a prototype limitation; the random-write amplification is a property of whole-node rewriting under the current buffer/fanout configuration and is the limitation a production deployment would hit first (§5.6). Reducing it requires either larger child buffers relative to the root (so a flush touches fewer children) or partial-node rewrites, both left to future work.

## 5.4 Read Latency

Point reads are GravTree’s strongest result (Table 2):  $2.6\times$  lower p99,  $3.5\times$  lower p99.99, and  $2.4\times$  higher throughput on a dataset  $30\times$  larger than the write

Table 2: Point read latency ( $\mu\text{s}$ ) and throughput, 1M-key dataset, 10M random lookups. **Bold** = better.

|          | p50         | p99         | p99.9       | p99.99      | Mops/s       |
|----------|-------------|-------------|-------------|-------------|--------------|
| GravTree | <b>1.47</b> | <b>2.42</b> | <b>9.29</b> | <b>14.4</b> | <b>0.620</b> |
| RocksDB  | 3.97        | 6.21        | 13.39       | 50.7        | 0.258        |
| Ratio    | $2.7\times$ | $2.6\times$ | $1.4\times$ | $3.5\times$ | $2.4\times$  |

Table 3: Peak write latency per 20 000-operation window (500 windows per workload), all in  $\mu\text{s}$ : range of per-window peaks over the run, count of windows whose peak exceeds  $100\mu\text{s}$ , and the single worst window. Both engines’ final-window spike is the explicit `flush()` issued after the last operation and is excluded. <sup>‡</sup>ops 9.96M–9.98M.

| Workload  | Engine   | Range    | >100 | Worst             |
|-----------|----------|----------|------|-------------------|
| SeqWrite  | GravTree | 0.2–1.4  | 0    | 1.4               |
|           | RocksDB  | 0.6–2.7  | 0    | 2.7               |
| RandWrite | GravTree | 0.3–4.2  | 0    | 4.2               |
|           | RocksDB  | 0.8–6.7  | 1    | 1060 <sup>‡</sup> |
| Mixed     | GravTree | 0.2–8.7  | 0    | 8.7               |
|           | RocksDB  | 0.6–15.3 | 0    | 15.3              |

buffer, so the result is not a DRAM-hit artefact. A GravTree lookup walks a tree of height 2–3, consulting one Bloom filter per node on the path and reading at most one leaf; there is no level-by-level probing of overlapping SSTables and no block cache indirection.

## 5.5 Latency Time-Series

Table 3 summarises 500 per-window peaks per workload. GravTree exhibits no mid-run window above  $100\mu\text{s}$  on any workload: its peak stays within 0.2– $1.4\mu\text{s}$  for SeqWrite and rises smoothly from 0.3 to  $4.2\mu\text{s}$  over the RandWrite run as leaves fill and the tree deepens. RocksDB is likewise flat for SeqWrite and Mixed at this scale; the *multi-level compaction cascades* that produce tens-of-millisecond stalls in production did not manifest on this hardware at  $N = 10\text{M}$  with 1.2GB of data, and we do not claim to have observed them. The one RocksDB excursion is a write stall of  $\approx 1\text{ms}$  in the final pre-flush RandWrite window (ops 9.96M–9.98M), which accounts for its p99.9/p99.99 in Table 1.

The final-window spike (excluded above) is the benchmark’s closing `flush()`. For GravTree it drains the root buffer through every level of the tree in one synchronous call—1.2s (SeqWrite) to 29.7s (RandWrite)—and is the clearest single measurement of the write amplification discussed above. For

RocksDB it is a memtable flush of 4–11 ms. Neither is an operational stall, but the GravTree figure shows that a bounded *per-event* cost does not imply a small *total* cost when many events are chained.

## 5.6 Discussion and Limitations

**Write throughput and amplification.** The dominant limitation of the current prototype is write throughput: 0.067 Mops/s sequential and 0.003 Mops/s random, versus 1.44 and 0.23 for RocksDB, driven by a measured write amplification of  $180\times$  at 10M random keys ( $9.4\times$  sequential) against  $10\times$  and  $1.0\times$  for RocksDB. The LocalFlush bound guarantees that no single compaction event exceeds  $O(K \cdot B)$  I/O, but it does not reduce total I/O: each key is rewritten once per tree level plus once per LocalFlush that touches its leaf, and the flush thread executes these rewrites serially under the tree lock, so a writer that outruns it is blocked at the hard limit. Three engineering changes, none of which alter the LocalFlush invariant, would close most of this gap and are the immediate next step: (i) parallel LocalFlush across disjoint children, which are independent by construction; (ii) delta files for internal nodes, so a flush into a child appends a run rather than rewriting the node; and (iii) larger leaves with a smaller root buffer, trading a modest increase in per-event bound for far fewer rewrites. Until these are implemented, GravTree is suited to read-heavy or latency-sensitive workloads with moderate write rates, not to bulk ingest.

**What the evaluation does and does not show.** It shows that GravTree’s write tail is tightly bounded ( $p_{99.99} < 20 \mu\text{s}$  on every workload) and that its p99 is lower than RocksDB’s on every workload. It does *not* show RocksDB compaction cascades: at 10M operations and 1.2 GB on a server-class NVMe device, RocksDB’s mid-run peaks stayed under  $100 \mu\text{s}$ . Larger datasets, larger values, or slower devices would be needed to test the “stall elimination” hypothesis directly, and we leave that as an explicit open question rather than a claim.

**No WAL; durability parity.** The prototype does not persist the write buffer to a WAL, and RocksDB was run with `disableWAL` to match. Absolute RocksDB write latencies with its default WAL-enabled configuration are higher (its p50 was  $\approx 3 \mu\text{s}$  in our original WAL-enabled 100K measurements), so the comparison here is conservative toward RocksDB. Adding a WAL to GravTree is orthogonal to LocalFlush but would add the same per-write cost.

**Single-threaded writers.** Concurrent external

writers are serialised by `write_buf_mu`; the numbers above are single-client. Per-shard skip lists are straightforward future work.

**Benchmark platform.** Measurements were taken on a cloud VM with a virtualised NVMe disk; absolute values will differ on bare metal, and the relative results should be re-validated there.

## 6 ZNS SSD Compatibility

Zoned Namespace (ZNS) SSDs [3] eliminate the device-level FTL by requiring applications to write each zone sequentially and reset zones before reuse. This shifts GC responsibility to the host software. Storage engines designed for random-overwrite block devices require substantial rearchitecting for ZNS compatibility. ZenFS [2] adds ZNS support to RocksDB by mapping SST files to zones, but compaction still rewrites multi-gigabyte zones non-sequentially within the RocksDB level hierarchy.

GravTree’s immutable node model aligns with ZNS requirements by construction:

- **Sequential write.** Each SSTable file is written front-to-back in a single pass (magic, header, Bloom filter, entries, sparse index, CRC) and never modified. This matches the ZNS per-zone sequential write requirement exactly.
- **Natural GC.** When LocalFlush creates new child files, the old child files are atomically invalidated. The corresponding zones can be immediately reset and recycled, providing deterministic, bounded GC with no separate GC thread.
- **Bounded zone pressure.** A single LocalFlush activates at most  $K + 1$  zones simultaneously (one per new child file, plus the parent). This simplifies zone allocation compared to LSM compaction, which may open  $O(|L_i| + |L_{i+1}|)$  files concurrently.

A ZNS backend using the libzbd API [15] is in development. We plan to report quantitative results on ZNS hardware in a follow-up evaluation.

## 7 Related Work

**LSM-tree optimisations.** A large body of work optimises LSM-tree compaction policy without bounding its scope. Dostoevsky [8] and Monkey [7] tune Bloom filter allocations across levels for different read/write trade-offs. WiscKey [12] separates

keys from values to reduce compaction I/O. PebblesDB [14] introduces guards to partition levels and reduce write amplification. None of these approaches bound per-compaction I/O to a constant independent of dataset size.

**B<sup>ε</sup>-trees.** Bender et al. [1] introduced the B<sup>ε</sup>-tree; TokuDB (now Percona TokuMX) commercialised it. SplinterDB [6] achieves high throughput with small per-flush scopes. GravTree uses the same parent-local flush insight but commits to immutable SSTable files as nodes, enabling consistent per-node Bloom filters and direct ZNS zone mapping without adaptation.

**NVMe-optimised LSM-trees.** SpanDB [5] places the WAL and top LSM levels on fast NVMe, reducing write latency. These are orthogonal to GravTree’s compaction-locality property and could be combined.

**ZNS-aware storage.** ZenFS [2] adapts RocksDB for ZNS. F2FS and dm-zoned provide ZNS-friendly file systems. None structurally guarantee that a single compaction touches a bounded number of zones; GravTree does.

## 8 Conclusion and Future Work

We presented GravTree, a tree-structured key-value store whose central invariant—LocalFlush touches exactly one parent and its direct children—bounds per-compaction I/O to a constant independent of dataset size. A C++17 prototype evaluated at 10M operations against RocksDB under equal (no-WAL) durability achieves 1.3–2.6× lower p99 latency on all four workloads, keeps p99.99 write latency below 20 μs, and delivers 2.4× higher point-read throughput. The cost is write throughput: whole-node rewrites give a measured write amplification of 9.4× (sequential) and 180× (random) at 10M keys versus 1.0× and 10× for RocksDB, and, with a single-threaded flush, a 21–80× lower sustained write rate. We did not observe LSM compaction cascades in RocksDB at this scale, so bounded-cascade behaviour remains an architectural argument rather than an empirical one.

Planned future work:

1. **Write-amplification reduction.** Partial-node rewrites or level-dependent buffer sizes so a random-key flush does not rewrite the whole subtree; parallel flush of sibling children with merge, Bloom-filter construction and CRC-64 moved off the tree lock.
2. **Write-ahead log.** Adding crash durability for

the in-memory write buffer.

3. **ZNS backend.** Quantitative evaluation on ZNS hardware with the libzbd backend.
4. **Bare-metal and slow-device revalidation.** Repeating the evaluation on dedicated hardware and on devices where LSM compaction is I/O-bound, where cascade stalls are expected to appear.
5. **Formal analysis.** Proof of the write-amplification bound and comparison with the B<sup>ε</sup>-tree optimality result of Bender et al.

The GravTree prototype is available as open-source software under the AGPL license.

## References

- [1] Michael A Bender, Martin Farach-Colton, Jeremy T Fineman, Yonatan R Fogel, Bradley C Kuszmaul, and Jelani Nelson. An introduction to B<sup>ε</sup>-trees and write-optimization. In *login: The USENIX Magazine*, volume 35, 2010.
- [2] Matias Bjorling. ZenFS: A storage backend for RocksDB on ZNS SSDs. <https://github.com/westerndigitalcorporation/zenfs>, 2021.
- [3] Matias Bjorling, Abutalib Aghayev, Hans Holmberg, Aravind Ramesh, Geert Du Plessis, Rafael Batista Dowsley, and Zvika Guz. Don’t be a blockhead: Zoned namespaces make work on conventional SSDs obsolete. In *USENIX HotStorage*, 2021.
- [4] Zhichao Cao, Siying Dong, Sagar Vemuri, and David HC Du. Characterizing, modeling, and benchmarking RocksDB key-value workloads at Facebook. In *18th USENIX Conference on File and Storage Technologies (FAST)*, 2020.
- [5] Hao Chen, Chaoyi Luo, Yuhao Wang, Shiwen Zhang, Yinan Su, Wenqing Wu, Qing Li, and Yuhao Chen. SpanDB: A fast, cost-effective LSM-tree based KV store on hybrid storage. In *19th USENIX Conference on File and Storage Technologies (FAST)*, 2021.
- [6] Alex Conway, Abhishek Gupta, Vijay Chidambaram, Martin Farach-Colton, Richard Spillane, Michael A Bender, and Rob Johnson. SplinterDB: Closing the bandwidth gap for NVMe key-value stores. In *USENIX Annual Technical Conference (ATC)*, 2020.

- [7] Niv Dayan, Manos Athanassoulis, and Stratos Idreos. Monkey: Optimal navigable key-value store. In *ACM SIGMOD International Conference on Management of Data*, 2017.
- [8] Niv Dayan and Stratos Idreos. Dostoevsky: Better space-time trade-offs for LSM-tree based key-value stores via adaptive removal of superfluous merging. In *ACM SIGMOD International Conference on Management of Data*, 2018.
- [9] Jeff Dean and Sanjay Ghemawat. LevelDB. <https://github.com/google/leveldb>, 2011.
- [10] Facebook. RocksDB: A persistent key-value store for fast storage environments. <https://rocksdb.org>, 2013.
- [11] Adam Kirsch and Michael Mitzenmacher. Less hashing, same performance: Building a better Bloom filter. *Random Structures & Algorithms*, 33(2):187–218, 2008.
- [12] Lanyue Lu, Thanumalayan Sankaranarayanan Pillai, Hariharan Gopalakrishnan, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. WiscKey: Separating keys from values in SSD-conscious storage. In *USENIX Conference on File and Storage Technologies (FAST)*, 2016.
- [13] Patrick O’Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O’Neil. The log-structured merge-tree (LSM-tree). *Acta Informatica*, 33(4):351–385, 1996.
- [14] Pandian Raju, Rohan Kadekodi, Vijay Chidambaram, and Ittai Abraham. PebblesDB: Building key-value stores using fragmented log-structured merge trees. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2017.
- [15] Western Digital. libzbd: A library to control zoned block devices. <https://github.com/westerndigitalcorporation/libzbd>, 2020.